

第11章 反射与动态特性

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 Class 对象:Java 里"类型本身"也是对象
- 掌握反射读取:获取字段、方法、构造器,读取注解
- 掌握反射调用:invoke、构造对象、访问私有成员
- 掌握动态代理 Proxy:在调用前后插入逻辑
- 理解"反射 + 注解 = 框架基座"的底层原理
- 理解反射的代价与安全边界,知道什么时候别用反射

💡 从 C 到 Java:程序认识自己

C 程序在运行时对自己一无所知:没有"这个结构体有哪些字段"的查询 API,没有"这个函数能不能被调用"的运行时检查——一切在编译期定死。Java 有 **反射(Reflection)**:程序在运行时可以检查自己的类结构、读取注解、调用任意方法、创建任意对象。C 里最接近的体验是:用函数指针表 + 字符串映射模拟"按名字找函数"——而反射是这门手艺的语言级完整版。

为什么生产代码需要它?因为框架必须"不认识你的类也能操作你的类":JSON 序列化库拿到任意对象要列出所有字段;ORM(MyBatis/Hibernate)要把任意对象映射成 SQL 参数;依赖注入(Spring)要按注解找到字段并注入;测试框架要运行任意类的 @Test 方法。这些库在编译期根本不知道你的类存在——它们全靠反射在运行时发现与操作。反射 + 注解(第12章)是全部 Java 框架的地基。

动态代理是反射的"行动版"。它让框架"包一层壳"接住你对接口的调用:AOP(事务、日志、权限)、RPC 的远程调用、Mock 测试——都是在壳里做的。Proxy.newProxyInstance 十几行就能实现"所有方法调用都先打日志"的拦截器,C 里等价物是手写一层 dispatch 表。

代价必须正视。反射调用比直接调用慢一个数量级(JDK 有优化缓存,但性能敏感路径仍要避免);setAccessible(true) 能突破封装——这是权力的诱惑,也是安全的裂缝。本章最后给出"何时用、何时绝不用"的清单。

学习顺序:Class 对象(\$11.1)→ 读写结构(\$11.2-11.3)→ 动态代理(\$11.4)→ 框架原理(\$11.5)→ 代价与安全(\$11.6)。

🧰 前置知识自查

- 第6章:类、封装、getter/setter;第10章:泛型(反射 API 大量泛型)
- 第4章:lambda(动态代理的 InvocationHandler 可用 lambda)
- 第12章将讲注解——本章先用"注解是什么"的最小直觉(@Override 那种)

🚀 本章学习路线

1. **第一阶段(约 1.5 小时)**:§11.1-11.2,亲手列出"任意类的全部字段与方法"
2. **第二阶段(约 1.5 小时)**:§11.3-11.4 反射调用与动态代理,实现一个日志拦截器
3. **第三阶段(约 1.5 小时)**:§11.5 框架原理 + §11.6 代价安全,完成章末实验
4. **验收标准**:能用反射把任意对象转成 Map;能写动态代理日志器;能说出三个"别用反射"的场景

本章知识导图

- §11.1 Class 对象:类型的类型
- §11.2 读取类结构:字段 / 方法 / 注解
- §11.3 反射调用:invoke / 构造 / setAccessible
- §11.4 动态代理:Proxy 与 InvocationHandler
- §11.5 反射 + 注解 = 框架基座
- §11.6 代价与安全边界

§11.1 Class 对象:类型的类型

11.1.1 三种拿法

每个类在 JVM 里对应一个 `Class<?>` 对象(第5章的元空间就是存它)。三种拿法:① 实例.`getClass()`;② 类字面量 `String.class`;③ `Class.forName("全限定名")`——只有它不需要编译期知道类型,这是框架们"按字符串操作类型"的入口。

示例 11-1 Class 对象三拿法

```
public class ClassObjectDemo {
    static class Student {
        private String name;
        private int score;
    }

    public static void main(String[] args) throws Exception {
        // 拿法 1:实例的 getClass
        Student s = new Student();
        Class<?> c1 = s.getClass();

        // 拿法 2:类字面量(编译期确定)
        Class<Student> c2 = Student.class;

        // 拿法 3:字符串加载(运行期才知道类名—框架的标准姿势)
        Class<?> c3 = Class.forName("ClassObjectDemo$Student");
        // 注意:嵌套类的全限定名是 外层$内层

        System.out.println(c1 == c2 && c2 == c3);    // true:同一个 Class 对象
        System.out.println("类名: " + c1.getName()); // ClassObjectDemo$Student
        System.out.println("简单名: " + c1.getSimpleName()); // Student
        System.out.println("父类: " + c1.getSuperclass());    // class java.lang.Object

        // 一个类只有一个 Class 对象:反射拿到的就是它
        Object created = c1.getDeclaredConstructor().newInstance();
        System.out.println(created instanceof Student);    // true
    }
}
```

★ 重点:Class 对象 = 类的"体检报告"

Class 对象上全是"查询接口":`getName/getSuperclass/getInterfaces/getDeclaredFields/getDeclaredMethods/getAnnotations`。C 的 `sizeof/offsetof` 只给布局;反射给的是完整的结构清单 + 运行时操作能力。这也解释了第5章为什么元空间装着"类元数据"——反射读的就是它。

§11.2 读取类结构:字段、方法与注解

示例 11-2 用反射“解剖”任意类

```
import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.lang.reflect.Modifier;

public class InspectDemo {
    static class Student {
        public String name;           // 公有字段
        private int score;           // 私有字段

        public void setScore(int s) { this.score = s; }

        private void secret() { }    // 私方法
    }

    public static void main(String[] args) {
        Class<?> c = Student.class;

        System.out.println("== 所有字段(含私有,不含继承)==");
        for (Field f : c.getDeclaredFields()) {
            System.out.println(" " + Modifier.toString(f.getModifiers())
                               + " " + f.getType().getSimpleName() + " " + f.getName());
        }

        System.out.println("== 所有方法(含私有)==");
        for (Method m : c.getDeclaredMethods()) {
            System.out.println(" " + m.getName() + "("
                               + m.getParameterCount() + " 参数) 返回 "
                               + m.getReturnType().getSimpleName());
        }

        System.out.println("== 按名字精确找 ==");
        try {
            Field score = c.getDeclaredField("score"); // 名字错了抛 NoSuchFieldException
            System.out.println(" 找到字段: " + score.getName());
            Method m = c.getDeclaredMethod("secret"); // 名字+参数类型
            System.out.println(" 找到方法: " + m.getName());
        } catch (NoSuchFieldException | NoSuchMethodException e) {
            System.out.println(" 没找到: " + e);
        }
    }
}
```

⚠ 易错点 1: getFields 与 getDeclaredFields 的区别

getFields 只返回 public(含父类继承的);getDeclaredFields 返回本类声明的全部(含 private,不含父类)。同理 getMethods/getDeclaredMethods。框架代码里 90% 用 getDeclaredFields + setAccessible——因为要操作私有字段。区分记法: "Declared = 自己声明的一切,不管访问级别;无 Declared = 公开可见的一切,含继承"。名字含 checked 异常(NoSuchFieldException 等),必须处理。

§11.3 反射调用:invoke 与 setAccessible

示例 11-3 反射读写字段、调用方法

```

import java.lang.reflect.Field;
import java.lang.reflect.Method;

public class InvokeDemo {
    static class Student {
        private String name = "张三";
        private int score = 60;

        public void addBonus(int bonus) { score += bonus; }

        @Override
        public String toString() { return name + ":" + score; }
    }

    public static void main(String[] args) throws Exception {
        Student s = new Student();

        // 读私有字段:先 getDeclaredField,再 setAccessible(true)
        Field name = Student.class.getDeclaredField("name");
        name.setAccessible(true);           // 突破 private(见 §11.6 代价)
        System.out.println("反射读 name: " + name.get(s));    // 张三

        // 写私有字段
        name.set(s, "李四");
        System.out.println("反射写 name 后: " + s);           // 李四:60

        // 调用公有方法:getMethod(名字, 参数类型...)
        Method addBonus = Student.class.getMethod("addBonus", int.class);
        Object result = addBonus.invoke(s, 15);              // 返回值;void 返回 null
        System.out.println("invoke 后: " + s);               // 李四:75 (result=null)

        // 调用私有方法
        Method secret = Student.class.getDeclaredMethod("secret");
        secret.setAccessible(true);
        // secret.invoke(s);    // 也能调,但"生产代码调私有方法"是设计坏味道

        // 用反射创建对象(替代 new;框架不认识你的类时用它)
        Student s2 = Student.class.getDeclaredConstructor().newInstance();
        System.out.println("反射新建: " + s2);              // 张三:60
    }
}

```

★ 重点:invoke 的返回值与异常包装

invoke 返回 Object,void 方法返回 null,基本类型自动装箱。被调方法抛出的任何异常,invoke 都会包装成 InvocationTargetException 再抛——要用 e.getCause() 拿真正的异常,直接看 e 的堆栈会一头雾水(堆栈指向 invoke 而非业务代码)。这是反射调试的第一坑。

§11.4 动态代理:给接口套一层壳

11.4.1 Proxy 与 InvocationHandler

动态代理:运行期为一个接口生成实现类,所有方法调用都先经过 `InvocationHandler.invoke`。调用方拿到的对象是"代理",行为由你的处理器决定——可以在真正实现前后插入日志、鉴权、事务、延迟加载。C 的对应物:手工写一层"转发函数指针表",Java 是语言级生成。

■ 示例 11-4 日志代理:零侵入地记录所有调用

```

import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;

public class ProxyDemo {
    // 业务接口:真正的服务实现
    interface PaymentService {
        boolean pay(long orderId, double amount);

        double query(long orderId);
    }

    // 真实实现(不想动它的代码)
    static class PaymentServiceImpl implements PaymentService {
        @Override
        public boolean pay(long orderId, double amount) {
            return amount > 0; // 假装真的扣款成功
        }

        @Override
        public double query(long orderId) {
            return 99.5;
        }
    }

    // 日志处理器:任何方法调用先打日志再转发
    static class LogHandler implements InvocationHandler {
        private final Object target; // 被代理的真实对象

        LogHandler(Object target) { this.target = target; }

        @Override
        public Object invoke(Object proxy, Method method, Object[] args)
            throws Throwable {
            System.out.println("【代理】调用 " + method.getName()
                + ", 参数: " + java.util.Arrays.toString(args));
            Object result = method.invoke(target, args); // 转发给真实对象
            System.out.println("【代理】返回: " + result);
            return result;
        }
    }

    public static void main(String[] args) {
        // 生成代理:接口列表 + 处理器;返回的对象类型就是接口
        PaymentService service = (PaymentService) Proxy.newProxyInstance(
            PaymentService.class.getClassLoader(),
            new Class<?>[]{PaymentService.class}, // 要代理的接口
            new LogHandler(new PaymentServiceImpl()));

        service.pay(1001L, 88.0);
        service.query(1001L);
        // 输出:两次调用都被日志包裹,业务代码零改动
    }
}

```



工程建议:动态代理的三大现实应用

1. **AOP(面向切面)**:Spring 的事务/日志/安全注解,本质是代理在方法前后插逻辑(Spring 对接口用 JDK 代理,对类用 CGLIB 子类代理)
2. **RPC 远程调用**:客户端拿到"代理接口",invoke 里把参数序列化发到服务端——调用方感觉像本地调用(Feign、Dubbo 的原理)
3. **Mock 测试**:Mockito 用代理拦截接口调用,返回预设值,无需真实依赖

§11.5 反射 + 注解 = 框架基座

11.5.1 一个迷你映射器

框架模式三步走: ① 用注解(第12章)给类打标记; ② 反射扫描注解与字段; ③ 按标记执行通用逻辑。下面实现"对象 ↔ Map"的迷你映射器——JSON 库和 ORM 的核心动作就是它。注解先"借用"一个自定义注解(第12章会教如何定义)。

示例 11-5 迷你对象映射器(框架雏形)

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.util.HashMap;
import java.util.Map;

public class MiniMapper {

    // 自定义注解:字段的"列名"(第12章详解定义语法)
    @Retention(RetentionPolicy.RUNTIME) // 必须:反射才能读到
    @Target(ElementType.FIELD)           // 只能标字段
    @interface Column {
        String value();
    }

    // 业务类:字段打上注解
    static class Student {
        @Column("stu_id")
        private String id;
        @Column("stu_name")
        private String name;
        private int score; // 没注解的字段:跳过

        Student(String id, String name, int score) {
            this.id = id;
            this.name = name;
            this.score = score;
        }

        // 需要 getter/setter 的"无参构造 + setter"版本见扩展练习
        String getId() { return id; }
    }

    // 核心:对象 → Map(通用逻辑:不认识 Student 也能转)
    static Map<String, Object> toMap(Object obj) throws Exception {
        Map<String, Object> map = new HashMap<>();
        for (Field f : obj.getClass().getDeclaredFields()) {
            Column col = f.getAnnotation(Column.class); // 反射读注解
            if (col == null) {
                continue; // 没注解就不导出
            }
            f.setAccessible(true);
            map.put(col.value(), f.get(obj)); // 注解值当键
        }
        return map;
    }

    public static void main(String[] args) throws Exception {
        Student s = new Student("2024001", "张三", 88);
        System.out.println(toMap(s));
        // {stu_id=2024001, stu_name=张三}:score 因无注解被跳过
    }
}
```

★ 重点:框架的"不认识你"是怎么成立的

toMap 的形参是 Object——它编译期不认识 Student,运行期通过 `getClass().getDeclaredFields()` 发现字段,通过 `getAnnotation` 发现契约,通过 `setAccessible + get` 读取值。你新增任何类、打上注解,这个"通用逻辑"立即生效,一行框架代码不用改。这就是 Spring/MyBatis/Jackson 与你写的业务代码解耦的全部秘密。框架 = 通用反射逻辑 + 注解契约 + 你的业务类。

§11.6 代价与安全边界

问题	说明	对策
性能	反射调用慢一个数量级(解析、装箱、安全检查)	热路径不用反射;框架内部有缓存(JVM 的反射缓存、MethodHandle)
可读性	类型错误延迟到运行期;IDE 找不到调用关系	封装成框架/工具;业务代码不裸写反射
封装破坏	<code>setAccessible(true)</code> 能读私有、改 final	只在自己代码里用;别对第三方库的对象乱开
模块化冲突	JPMS 下模块可拒绝反射(第13章)	模块开放 open 或显式授权

⚠ 易错点 2:三个"别用反射"的场景

1. 能用接口/多态解决,别用反射:直接调用快、类型安全、可读(反思:框架才需要"不认识你")
2. 性能敏感的循环体内,别用反射:十亿级调用场景,缓存 Method 对象或换编译期方案
3. 依赖"私有成员"做业务,别用反射:那是在补封装设计的洞,正确做法是修类的设计

示例 11-6 性能对策:缓存反射对象

```
import java.util.concurrent.ConcurrentHashMap;

public class ReflectCache {
    // 反射查找很贵(解析、安全检查);缓存后只查一次
    private static final ConcurrentHashMap<String, java.lang.reflect.Method>
        CACHE = new ConcurrentHashMap<>();

    static java.lang.reflect.Method cachedMethod(String className,
        String methodName) throws Exception {
        return CACHE.computeIfAbsent(className + "#" + methodName, key -> {
            try {
                return Class.forName(className).getMethod(methodName);
            } catch (ReflectiveOperationException e) {
                throw new IllegalArgumentException("反射失败: " + key, e);
            }
        });
    }

    public static void main(String[] args) throws Exception {
        // 首次查找后,后续调用直接命中缓存
        var m1 = cachedMethod("java.lang.String", "length");
        var m2 = cachedMethod("java.lang.String", "length");
        System.out.println(m1 == m2);    // true:同一个 Method 对象
    }
}
```

本章速查表

主题	要点
Class 对象	类型也是对象;三种拿法:getClass() / 类.class / Class.forName("全限定名")
嵌套类名字	全限定名是 外层\$内层(如 ClassObjectDemo\$Student)
字段	getDeclaredFields 全部(含私有)/ getFields 仅 public(含继承)
方法	getDeclaredMethod(名字, 参数类型...) 精确找;NoSuchMethodException 是 checked
setAccessible	突破 private 访问;用前先 getDeclared;只对自己代码用
invoke	返回 Object;void 返回 null;被调异常包成 InvocationTargetException,取 getCause()
新建对象	getDeclaredConstructor().newInstance();需要无参构造器
动态代理	Proxy.newProxyInstance(加载器, 接口数组, InvocationHandler);调用全部经过 invoke
代理应用	AOP 事务日志 / RPC 远程调用 / Mock 测试
框架模式	注解标记 → 反射扫描 → 通用逻辑;Object 形参 + getDeclaredFields 发现一切
性能	反射慢一个数量级;热路径禁用;框架用缓存弥补
安全	setAccessible 是权力也是裂缝;JPMS 可封禁反射(第13章)

最小必会清单

- 能说出 Class 对象的三种拿法与各自的编译期依赖
- 能列出"任意类的所有字段(含私有)及其类型"的代码
- 能写出反射读写字段、调用方法的完整代码(含 setAccessible 与异常处理)
- 能解释 InvocationTargetException 的包装与 getCause
- 能写出动态代理日志拦截器,并说出三大应用场景
- 能解释"反射 + 注解 = 框架基座"的三步模式
- 能说出三个"别用反射"的场景与性能对策

自测题

Q 1. Class.forName 与 类.class 的区别是什么?框架为什么用 forName?

✓ 参考答案

类.class 在编译期确定,编译器就知道类型;Class.forName 接收字符串,运行期才解析——字符串可以来自配置文件、用户输入、注册表。框架在编译期不认识你的业务类,只能拿到"类名字字符串"运行时加载。代价:名字写错、类不存在,错误发生在运行期(ClassNotFoundException)。

Q 2. 反射调用一个抛异常的方法,为什么 catch 到的是 InvocationTargetException?怎么拿真正的异常?

✓ 参考答案

invoke 为了不吞掉被调方法的异常,把任何异常都包装进 InvocationTargetException 抛给调用方(这样 invoke 的签名才能是"只抛 ReflectiveOperationException 家族")。真正异常在 e.getCause() 里,排查日志时要打印 getCause 的完整堆栈,否则只看到 invoke 一层的栈,找不到业务错误点。

Q 3. getDeclaredFields 拿到的字段名,能不能直接用于 setAccessible + get?有哪些前置条件?

✅ 参考答案

可以。前置:① 字段确实在该类声明(不在父类——父类字段要 `getSuperclass` 递归找);② `setAccessible(true)` 成功(JPMS 未封禁、安全管理器允许);③ 静态字段 `get(null)`、实例字段 `get(实例)`。另外 `getDeclaredFields` 不保证顺序——依赖顺序的逻辑必须显式排序(如按注解的 `order` 值)。

Q 4. 动态代理为什么只能代理接口,不能代理类?

✅ 参考答案

JDK 代理生成的是“实现接口的新类”——Java 单继承,无法继承任意目标类。代理类没有目标类的私有状态与 `final` 方法。要代理类(无接口的类)用 CGLIB/ByteBuddy(生成子类,字节码技术),但 `final` 类与 `final` 方法依然代理不了。选型:面向接口编程(第6章)的好处在这里兑现——接口是天然的代理边界。

Q 5. 用反射实现的 JSON 序列化,为什么比手写 `getter` 拼接慢?哪些环节慢?

✅ 参考答案

慢在三处:① 每次 `get` 要装箱(基本类型转 `Object`)与类型判断;② `setAccessible` 有安全检查(现代 JVM 有缓存优化);③ 方法分发走 `invoke` 的间接层。缓解:缓存 `Field/Method` 对象(不每次重新查找)、用 `MethodHandle`、或编译器在首次反射后生成直接调用的字节码(Jackson 的 `Afterburner`、Gson 的反射缓存)。热路径上,手写 `getter` 依然最快。

Q 6. 你的工具方法接受任意对象并打印其所有字段名与值。写出核心代码(注意私有字段与父类字段)。

✅ 参考答案

```
static void dumpFields(Object obj) throws IllegalAccessException {
    Class<?> c = obj.getClass();
    while (c != null) { // 沿继承链往上走
        for (Field f : c.getDeclaredFields()) {
            if (java.lang.reflect.Modifier.isStatic(f.getModifiers())) {
                continue; // 跳过静态字段
            }
            f.setAccessible(true);
            System.out.println(c.getSimpleName() + "." + f.getName()
                               + " = " + f.get(obj));
        }
        c = c.getSuperclass(); // 处理父类字段
    }
}
```

要点:while 循环沿 `getSuperclass` 处理父类字段(到 `Object` 为止);跳过静态字段;`setAccessible` 包住完整读取。生产等价物:Apache Commons 的 `FieldUtils.getAllFields`、Gson 的内部反射——理解原理后,工具库只是这几十行的工程化。

章末实验题:迷你对象映射器

实验 11:迷你对象映射器 + 日志代理

实验目标:编写两部分程序:① 对象 ↔ Map 的迷你映射器(含注解支持与类型转换);② 动态代理日志拦截器。综合运用本章全部知识点。

实验要求:

- 任务 1:定义注解 @Column(value, nullable),分别可标字段;nullable 默认 true(覆盖 §11.2 注解读取,注解定义语法见第12章预习)
- 任务 2:toMap(Object):只导出带 @Column 的字段,键用注解 value;私有字段正常读取(覆盖 §11.2-11.3 字段反射)
- 任务 3:fromMap(Class<T>, Map):按注解映射回对象,字段类型做基本类型转换(数字字符串转 int 等),null 且 nullable=false 时抛异常(覆盖 §11.3 构造与写入)
- 任务 4:字段映射支持继承:父类的 @Column 字段也处理(覆盖 §11.2 继承链)
- 任务 5:定义接口 Repository 与实现,用动态代理包装:每次调用前打印"调用 + 参数",调用后打印"耗时"(覆盖 §11.4 动态代理)
- 任务 6:代理对方法调用做"幂等保护":query 类方法(名字以 query 开头)失败自动重试 1 次(覆盖 §11.4 代理逻辑)
- 任务 7:main 里演示:Student 对象转 Map、Map 转回对象、代理调用,输出完整报告(覆盖全部)

实验环境:JDK 17+。

第一步 写注解与 toMap(最容易见效),跑通再写 fromMap 的类型转换

第二步 任务 4 与自测题 6 的思路一致:while (c != null) 沿父类走

第三步 任务 6 的"重试"在 InvocationHandler.invoke 里用循环包住 method.invoke

参考答案要点

```

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;
import java.util.HashMap;
import java.util.Map;

/** 迷你对象映射器 + 日志代理 — 第11章综合实验参考答案
 * 覆盖:注解定义与读取、字段反射、继承链扫描、类型转换、动态代理、重试
 */
public class MiniMapperLab {

    // 任务 1:字段注解
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    @interface Column {
        String value();

        boolean nullable() default true; // 默认允许 null
    }

    // 业务类:含父类字段(任务 4)
    static class BaseEntity {
        @Column("id", nullable = false)
        private Long id;
    }

    static class Student extends BaseEntity {
        @Column("stu_name")
        private String name;
        @Column("stu_score")
        private Integer score; // 用包装类型,便于 null 语义
        private String secret; // 无注解:不参与映射
    }

    // 任务 2:对象 → Map(沿继承链扫描)
    static Map<String, Object> toMap(Object obj) throws IllegalAccessException {
        Map<String, Object> map = new HashMap<>();
        for (Class<?> c = obj.getClass(); c != null; c = c.getSuperclass()) {
            for (Field f : c.getDeclaredFields()) {
                Column col = f.getAnnotation(Column.class);
                if (col == null) {
                    continue;
                }
                f.setAccessible(true);
                map.put(col.value(), f.get(obj));
            }
        }
        return map;
    }

    // 任务 3:Map → 对象(类型转换 + nullable 校验)
    static <T> T fromMap(Class<T> type, Map<String, Object> data) throws Exception {
        T obj = type.getDeclaredConstructor().newInstance(); // 需要无参构造
        for (Class<?> c = type; c != null; c = c.getSuperclass()) {
            for (Field f : c.getDeclaredFields()) {
                Column col = f.getAnnotation(Column.class);
                if (col == null || !data.containsKey(col.value())) {
                    continue;
                }
                f.setAccessible(true);
                Object raw = data.get(col.value());
                if (raw == null) {
                    if (!col.nullable()) {

```

```

        throw new IllegalArgumentException(
            "字段 " + f.getName() + " 不允许为 null");
    }
    f.set(obj, null);
    continue;
}
// 基本类型转换:字符串/数字 → 字段类型
Class<?> ft = f.getType();
Object converted = raw;
if (ft == Long.class || ft == long.class) {
    converted = Long.parseLong(String.valueOf(raw));
} else if (ft == Integer.class || ft == int.class) {
    converted = Integer.parseInt(String.valueOf(raw));
}
f.set(obj, converted);
}
}
return obj;
}

// 任务 5+6:代理接口与处理器
interface Repository {
    String queryById(long id);

    void save(String data);
}

static class RepositoryImpl implements Repository {
    @Override
    public String queryById(long id) {
        return "数据#" + id;
    }

    @Override
    public void save(String data) {
        System.out.println("    [真实保存] " + data);
    }
}

static class LoggingHandler implements InvocationHandler {
    private final Object target;

    LoggingHandler(Object target) { this.target = target; }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args)
        throws Throwable {
        System.out.println("    [代理] 调用 " + method.getName()
            + " 参数 " + java.util.Arrays.toString(args));
        long start = System.nanoTime();
        int attempts = method.getName().startsWith("query") ? 2 : 1;
        Throwable last = null;
        for (int i = 0; i < attempts; i++) {
            try {
                Object result = method.invoke(target, args);
                long cost = (System.nanoTime() - start) / 1_000_000;
                System.out.println("    [代理] 返回 " + result + " 耗时 " + cost + "ms");
                return result;
            } catch (java.lang.reflect.InvocationTargetException e) {
                last = e.getCause();
                System.out.println("    [代理] 第 " + (i + 1)
                    + " 次失败: " + last.getMessage());
            }
        }
        throw last;
    }
}

public static void main(String[] args) throws Exception {

```

```

// 任务 7:完整演示
Student s = new Student();
s.id = 1001L;
s.name = "张三";
s.score = 88;
s.secret = "不导出";           // 无注解字段

Map<String, Object> map = toMap(s);
System.out.println("== toMap ==");
System.out.println(" " + map);   // {stu_score=88, stu_name=张三, id=1001}

Student back = fromMap(Student.class, map);
System.out.println("== fromMap ==");
System.out.println(" id=" + back.id + " name=" + back.name
    + " score=" + back.score + " secret=" + back.secret);

System.out.println("== 代理 ==");
Repository repo = (Repository) Proxy.newProxyInstance(
    Repository.class.getClassLoader(),
    new Class<?>[]{Repository.class},
    new LoggingHandler(new RepositoryImpl()));
repo.queryById(42);
repo.save("订单数据");
System.out.println("实验完成。");
    }
}

```

设计说明:任务 4 的继承链扫描(toMap/fromMap 里的 for 循环沿 getSuperclass 上行)同时覆盖了父类字段与"字段在哪个类声明"的可读性;任务 3 的类型转换只演示 Long/Integer 两条分支,实际框架是"类型转换器注册表"(String→LocalDate、byte[]→图片等),原理一致;任务 6 的重试策略放在代理里——业务代码完全无感,这正是 AOP 的核心价值。扩展方向:把 fromMap 支持构造器参数映射(第14章 record 天然适合)、加 @Transient 排除字段、用泛型 + Class<T> 参数把工具类改成类型安全版本。

下一章预告:第12章 注解与元编程。注解的定义、元注解、处理器的完整实现——把第11章"借用"的注解正式讲透。